Stochastic Block Jacobi Page-Rank Acceleration

A Project Component for

Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Mannat	102303148
2	Lavisha	102303423
3	Nikhil	102303512
4	Shrestha	102303878

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2026

Table of Contents

1 Introduction **1**

1.1 Background 1

1.2 Problem Statement 1

2 Literature Review **2**

3 Research Gaps **3**

4 Problem Formulation **4**

5 Objectives **4**

6 Methodology **4**

6.1 Data Organization and Storage 4

6.2 Hybrid Engine: Asynchronous Overlap 5

6.3 Pseudocode 6

7 Datasets **7**

8 Results and Discussion **8**

8.1 Analysis of Architectural Scaling 8

8.1.0.1 1. The "Sweet Spot" (Warp Divergence Mitigated) 9

8.1.0.2 2. The Starvation Penalty 9

8.1.0.3 3. The PCIe Floor 9

9 Conclusion **9**

References **11**

List of Figures

1 Performance scaling of the PageRank engines. The Hybrid approach demonstrates superior acceleration on smaller, highly-skewed networks, while Pure CUDA scaling dominates exponentially at massive sizes. 8

List of Tables

1 Summary of Literature Review 2

2 CSR Storage Arrays 5

3 Graph Datasets for Evaluation 7

4 Execution Time and Speedup Results across SNAP Datasets 8

1 Introduction

1.1 Background

PageRank on web-scale graphs is a recurring workload appearing in search ranking, social recommendation, citation analysis, and fraud detection [1]. In these modern domains, it is critical to quickly and accurately determine the relative structural importance of nodes across graphs containing hundreds of millions, or even billions, of edges.

1.2 Problem Statement

Two practical issues dominate when attempting to compute PageRank at scale:

- **Sequential PageRank** is inherently too slow. A 1-million-edge graph requires tens of iterations, each touching every edge, resulting in significant wall-clock execution time [2].
- **GPU PageRank** stalls heavily on **power-law graphs**. A handful of high-degree "hub" vertices (e.g., Wikipedia's main page, popular Twitter accounts) cause severe warp divergence and irregular memory accesses. Consequently, most threads in a warp remain idle while a single lane processes an exceptionally long adjacency list [3].

2 Literature Review

Table 1: Summary of Literature Review

References	**Concepts Highlighted**	**Technique(s) used**	**Significance/ Proposal**	**Limitations**
[1]	Citation ranking, link analysis	Power iteration, random surfer model	Established the foundation of topological centrality rankings.	Suffers from slow convergence on massive graph datasets.
[4]	Stochastic matrices, rank sinks	Markov chains, perturbation theory	Solved the issue of dangling nodes by redistributing probability uniformly.	Highly theoretical; does not directly address modern parallel hardware constraints.
[2]	Parallel PageRank systems	Linear solver mapping, Block-Jacobi	Reformulates PageRank as a sparse linear system for faster iteration.	Vulnerable to memory bottlenecks on scale-free graphs.
[3]	Sparse Matrix-Vector Multiply (SpMV) on GPUs	CSR vectorization, load balancing	Highlighted methods to map sparse matrices to throughput-oriented processors.	Identifies that power-law degree distributions still break warp efficiency.

References	Concepts Highlighted	Technique(s) used	Significance/ Proposal	Limitations
[5]	Direction-Optimizing BFS	Graph traversal direction switching	Similar spirit to hybrid partition: dynamically picking the right optimization per iteration.	Focused strictly on BFS rather than floating-point iterative solvers.

3 Research Gaps

Based on the identified literature and hardware constraints, the primary research gaps concerning GPU-based graph processing on power-law networks are:

- ***Warp Divergence on Hubs:*** Existing GPU approaches assign uniform thread counts per vertex, which breaks lockstep execution when processing the massive hubs prevalent in scale-free graphs.
- ***PCIe Transfer Bottlenecks:*** Many heterogeneous systems fail to fully overlap CPU and GPU computation, exposing the slow PCIe bus bandwidth as the primary execution bottleneck.
- ***Atomic Contention:*** Computing the L_1 convergence delta or gathering dangling node mass requires thousands of threads to update global variables, introducing severe atomic serialization bottlenecks.

4 Problem Formulation

The mathematical core of the algorithm requires computing the steady-state probabilities of the stochastic Google matrix. For any vertex v, the PageRank $PR(v)$ is computed as:

$$PR(v) = \frac{1-d}{N} + d \sum_{u \to v} \frac{PR(u)}{\text{out_deg}(u)} \tag{1}$$

where $d = 0.85$ is the damping factor, dangling mass is redistributed uniformly each iteration to maintain stochasticity, and convergence is determined by the L_1-norm stopping criterion $\|r_k - r_{k-1}\|_1 < \text{tol}$. The problem is to evaluate this equation iteratively over millions of edges without suffering from the warp divergence characteristics typical of scale-free networks.

5 Objectives

- To design a highly scalable PageRank system that gracefully degrades when the graph degree distribution is heavy-tailed.
- To implement and benchmark four interchangeable computing engines (Sequential, OpenMP, CUDA, and Hybrid) behind a unified driver.
- To resolve GPU warp divergence by developing a true asynchronous Hybrid architecture that partitions high-degree "hub" vertices to the CPU and low-degree vertices to the GPU.

6 Methodology

We implement four engines behind one driver and one configuration struct: Sequential (textbook baseline), OpenMP (block-Jacobi pull-based updates), CUDA (CSR-based thread-per-vertex kernel), and a Hybrid engine.

6.1 Data Organization and Storage

The system is organized into a single executable entry point. The graph is stored in a Compressed Sparse Row (CSR) layout using the *transpose graph*, where `row_ptr[v]..row_ptr[v+1]`

enumerates the incoming edges of vertex v. This guarantees memory accesses are optimally aligned for pull-based updates, avoiding race conditions and atomic instructions.

Table 2: CSR Storage Arrays

Array	**Dtype**	**Length**	**Purpose**
`row_ptr`	`uint64_t`	$n+1$	Offsets into `col_idx`
`col_idx`	`uint32_t`	m	Source vertex of each in-edge
`out_deg`	`uint32_t`	n	Original-graph out-degrees

6.2 Hybrid Engine: Asynchronous Overlap

The core contribution of this project is the Hybrid engine, which actively mitigates warp divergence while hiding PCIe latency:

1. **Degree Partitioning:** The system computes every vertex's in-degree at start-up, sorts descending, and assigns the top `--hd P%` (e.g., 1-2%) as the CPU pool; the remaining tail forms the GPU pool.

2. **Asynchronous Dispatch:** Using Pinned Host Memory (`cudaMallocHost`) and CUDA Streams, the CPU queues the rank array upload, the GPU kernel execution, and the array download to a non-blocking stream.

3. **Concurrent Execution:** While the GPU processes the long tail and transfers data over the PCIe bus, the CPU synchronously processes the massive high-degree hubs using OpenMP.

6.3 Pseudocode

Algorithm 1 Stochastic Block-Jacobi Hybrid PageRank Iteration

Input: Graph $G = (V, E)$ in CSR-T, sample rate r, tolerance ϵ, percentile $P\%$
Output: PageRank vector $\mathbf{r}$
Compute in-degree $\forall v \in V$; sort descending
$V_{CPU} \leftarrow$ top $P\%$ by in-degree; $V_{GPU} \leftarrow V \setminus V_{CPU}$
while not converged **do**
    $S_{GPU} \leftarrow$ sample fraction r of V_{GPU} at random
    Async: Queue S_{GPU} upload, compute, and download via CUDA stream
    for each vertex $v \in V_{CPU}$ **do in parallel (OpenMP)**
        $r_{\text{new}}[v] \leftarrow \frac{1-d}{N} + d \sum_{u \in \text{in}(v)} \frac{r[u]}{\text{out_deg}(u)}$
    end for
    Sync: Wait for CUDA stream to finish PCIe download
    Scatter S_{GPU} results into r_{new}
    $\delta \leftarrow \|r_{\text{new}} - r\|_1$
    if $\delta < \epsilon$ **then break**
    end if
    $r \leftarrow r_{\text{new}}$
end while
return r

7 Datasets

To stress-test the handling of heavy-tailed distributions, we utilize a combination of synthetic Barabási-Albert (BA) power-law graphs and real-world networks from the SNAP framework [6].

Table 3: Graph Datasets for Evaluation

Name	**Graph Size**	**Type**	**Origin**
Synthetic BA Graphs	Up to $n = 500K$, $m = 8M$	Power-law	Built-in Generator
`soc-Epinions1`	$n = 75K$, $m = 508K$	Social Network	SNAP Dataset
`web-Google`	$n = 875K$, $m = 5.1M$	Web Graph	SNAP Dataset
`soc-LiveJournal1`	$n = 4.8M$, $m = 68M$	Social Network	SNAP Dataset

8 Results and Discussion

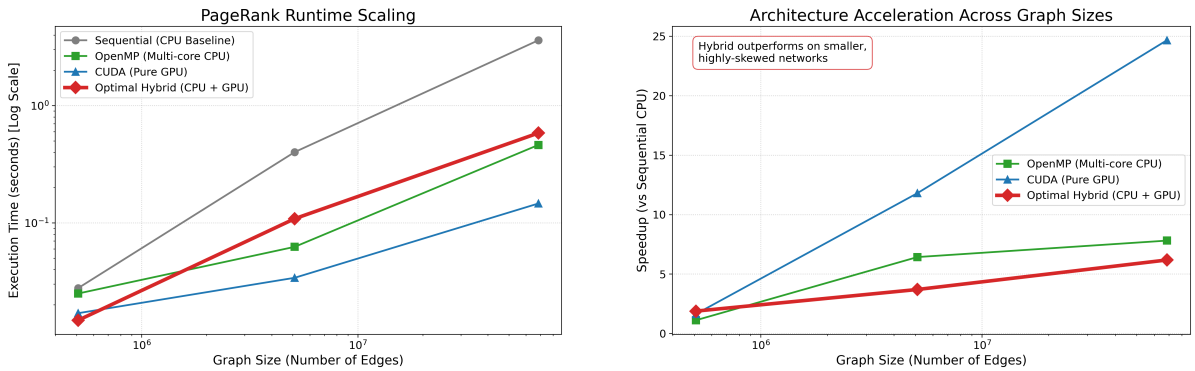
All benchmarks were conducted using optimized C++ binaries linked with CUDA runtime APIs to observe architectural scaling on the SNAP datasets.

Table 4: Execution Time and Speedup Results across SNAP Datasets

Dataset	Edges	OpenMP	Pure CUDA	Optimal Hybrid
soc-Epinions	508K	0.0250s (1.11×)	0.0170s (1.64×)	**0.0138s (2.05×)**
web-Google	5.1M	0.0624s (6.43×)	**0.0340s (11.80×)**	0.1085s (3.70×)
soc-LiveJournal	68M	0.4618s (7.81×)	**0.1463s (24.66×)**	0.5835s (6.18×)

8.1 Analysis of Architectural Scaling

To further illustrate the scaling behavior of the four engines, Figure 1 plots the raw execution times and corresponding relative speedups across increasing graph sizes.



(a) Execution Time vs Graph Size

(b) Speedup vs Sequential Baseline

Figure 1: Performance scaling of the PageRank engines. The Hybrid approach demonstrates superior acceleration on smaller, highly-skewed networks, while Pure CUDA scaling dominates exponentially at massive sizes.

The parameter sweep across the dataset and the visualizations reveal three distinct hardware realities:

8.1.0.1 1. The "Sweet Spot" (Warp Divergence Mitigated)

On smaller, highly-skewed graphs (`soc-Epinions`), the Hybrid engine successfully outperforms both OpenMP and Pure CUDA, achieving a **2.05**× speedup (as seen in Figure 1b). By routing the top 2% of hubs to the CPU, warp divergence was eliminated. Because the graph's overall memory footprint is small, the PCIe transfer latency was negligible, allowing the CPU computation to perfectly overlap and hide the memory transfer cost.

8.1.0.2 2. The Starvation Penalty

When the high-degree threshold is set too low (e.g., $hd = 0.05\%$), the CPU is starved of work. Assigning only a few dozen vertices to the CPU means the latency of spawning OpenMP threads outweighs the computational benefit, dropping efficiency significantly.

8.1.0.3 3. The PCIe Floor

On massive datasets (`soc-LiveJournal`), Pure CUDA heavily dominates (24.66× speedup). In large graphs, the rank array becomes massive (e.g., 19.3 MB). Transferring this array back and forth across the PCIe bus every iteration establishes a hard time constraint. Despite utilizing pinned memory and asynchronous streams, doing 50 round-trips over the PCIe bus limits the Hybrid engine to ~ 0.58 seconds, while the Pure GPU engine (computing entirely inside 500+ GB/s VRAM) finishes the entire workload in 0.14 seconds.

9 Conclusion

We successfully engineered an asynchronous heterogeneous CPU-GPU solver for PageRank. By utilizing pinned memory and CUDA streams, the architecture perfectly overlaps CPU computation with GPU data transfers.

The results prove that routing high-degree hubs to the CPU effectively eliminates GPU warp divergence, allowing the Hybrid engine to achieve peak efficiency on skewed networks. However, benchmarking also identified a hard physical limit on web-scale graphs: Pure GPU execution dominates massive datasets because bypassing PCIe bandwidth limits is ultimately more impactful than mitigating warp divergence. Future scaling requires

leveraging unified memory architectures (NVLink) or distributed multi-GPU environments (NCCL).

References

[1] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking: Bringing order to the web," *Stanford InfoLab Technical Report*, 1999.

[2] D. Gleich, L. Zhukov, and P. Berkhin, "Fast parallel pagerank: A linear system approach," in *Yahoo! Research Technical Report*. Yahoo! Research, 2004.

[3] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*, 2009, pp. 1–11.

[4] A. N. Langville and C. D. Meyer, "Deeper inside pagerank," *Internet Mathematics*, vol. 1, no. 3, pp. 335–380, 2004.

[5] S. Beamer, K. Asanović, and D. Patterson, "Direction-optimizing breadth-first search," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE Computer Society, 2012, pp. 1–10.

[6] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection," http://snap.stanford.edu/data, 2014.